Université Libanaise
Faculté de Science-II
Département Informatique

Project Management - I3330

Tasty Bites

Full Stack Food Ordering Web Application
Phase I — Project Definition & Agile Setup

Presented by : Serena Sakr ID:60260
Marilyne Antoun ID:61138
Rose Marie Antoun ID:61329

Presented To: Dr Mira Abboud

Table of Content

Part 01 — Project Vision & Scope Definition

1.1 System Need

In today's fast paced environment, customers want convenient digital solutions for ordering food online. Traditional phone-based ordering is slow, error-prone, and lacks transparency. At the same time, restaurant owners and administrators need a simple way to manage their menus, track incoming orders, and keep things running smoothly. Tasty Bites fills this gap by providing a full-stack web-based food delivery platform that lets customers browse menus, manage carts, place orders, provide delivery addresses, and leave feedback, while giving administrators the tools to update the menu and oversee all orders , all from a clean, modern interface.

1.2 Objectives

Business & User Objectives:
Allow customers to browse menu items and place food orders online from any location.
Provide a responsive and user friendly interface accessible on both desktop and mobile devices.
Enable customers to create accounts and securely log in to the system.
Allow users to save and manage delivery addresses linked to their accounts.
Provide a smooth and efficient checkout process for completing orders.
Enable customers to view order history and reorder previous items easily.
Allow customers to submit feedback, ratings, and reviews.

System & Functional Objectives:
Implements a menu management system for administrators (add, update, delete items).
Provides an admin dashboard to monitor and manage incoming orders.
Implements a search functionality to quickly find menu items.
Display promotions and special offers on the homepage to increase engagement.
Ensure secure authentication using password hashing mechanisms (e.g., bcrypt).
Support cart management including adding, updating, and removing items.

Technical & Project Objectives:
Develop the system using a modern web architecture (React frontend + ASP.NET Core backend).
Follow Agile methodology with structured sprint planning and delivery.
Maintain version control using GitHub with proper collaboration practices.
Ensure system reliability through testing and validation.
Provide clear and complete project documentation.

1.3 System Boundaries

In Scope: The system covers customer registration and login, menu browsing with category filtering and search, cart management (add, update, and remove items), checkout with payment information collection, delivery address management, customer feedback and testimonials, order history with reordering, an admin dashboard for menu management (full create, read, update, delete) and order oversight, a homepage carousel and promotional cards, responsive navigation, and an about page where visitors can learn about the restaurant.

Out of Scope: The system does not support multiple restaurants, native mobile apps, live GPS driver tracking, third-party payment gateway processing, real-time delivery notifications, or third-party logistics integration. These features go beyond the current project goals and may be considered in future versions.

1.4 Stakeholders

The system involves several stakeholders who interact with the platform and have different interests.

Stakeholder	Role	Interest / Expectations
Customer	End Users.Places orders online	Easy menu browsing, quick ordering, secure login, delivery address management, ability to leave feedback and view order history.
Restaurant Admin	Manages menu items and orders	Full control over menu items (add, edit, delete), visibility into incoming orders, order status management.
Restaurant Owner	Business stakeholder	More orders, customer insights through feedback, efficient order management.
Delivery Staff	Delivers orders to specified address	Accurate delivery address,clear order details,and an efficient delivery process.
Project Team. (Scrum roles)	Designs, builds, and maintains the system	Clear requirements, manageable sprints, hands-on Agile/Scrum experience, professional Git workflow.
Course Instructors	Evaluators	Professional process, proper documentation, working increments, disciplined Agile practices.
System Administrator	Maintains,monitors and support the system	System reliability, data security, error free performance and regular system maintenance and updates

1.5 Development Approach & Team Roles

The team follows Scrum as the Agile framework. Development is organized into time-boxed sprints (2 weeks each), with sprint planning, daily standups, sprint reviews, and retrospectives. The product backlog is continuously refined and prioritized using Planning Poker estimation.

The team follows a collaborative approach where each member contributes to all parts of the system while having a primary area of responsibility.

Member Name	Roles	Responsibilities
Marilyne Antoun	Product Owner, Full Stack Developer (primary frontend)	Defines and prioritizes backlog; builds React components, styling, and routing.
Serena Sakr	Scrum Master, Full Stack Developer (primary backend)	Facilitates Scrum ceremonies; builds ASP.NET Core API endpoints and authentication.
Rose Marie Antoun	Full Stack Developer (primary database,QA Tester)	Manages MySQL schema and deployment; writes and runs acceptance tests.

Part 02 — Product Backlog & Sprint Planning

2.1 Product Backlog

The following user stories form the product backlog, estimated using Planning Poker. Each story includes acceptance criteria written as user facing conditions and its priority.Story points were estimated using Planning Poker as a team based estimation technique. Each team member independently selected a value based on effort and complexity, and differences were discussed until consensus was reached.

ID	Epic	User Story Description	Acceptance Criteria	Priority	Story Points
US1	Auth	**User Registration** As a user, I want to register with my email and password so I can access the food ordering system.	1. User can enter email and password on a signup form. 2. System rejects registration if email is already taken. 3. System rejects weak or empty passwords with feedback. 4. On success, confirmation message is shown and user can log in.	Must Have	5

US2	Auth	**User Login** As a registered user, I want to log in with my credentials so I can access my account and place orders.	1. User can enter email and password on a login form. 2. System grants access on valid credentials. 3. Error message for wrong email or password without revealing which. 4. After login, user is returned to the page they were on.	Must Have	5
US3	Auth	**User Logout** As a logged-in user, I want to log out so my session ends and my account stays secure.	1. Logout option visible only when logged in. 2. After logout, user cannot access account specific pages. 3. Cart resets to zero on logout. 4. Login option returns in the navigation bar.	Must Have	2
US4	Cart	**Add Item to Cart** As a customer, I want to add items to my cart with a selected quantity so I can build my order.	1. User must be logged in to add items; otherwise a login prompt is shown. 2. User can select a quantity before adding an item. 3. A success message is shown after an item is added. 4. If the same item is added again, quantity updates instead of creating a duplicate.	Must Have	8
US5	Cart	**View Shopping Cart** As a customer, I want to view all items in my cart so I can review my order before checkout.	1. Cart page shows all items the user has added. 2. Each item shows its name, image, unit price, quantity, and subtotal. 3. Total order price is calculated and shown at the bottom. 4. If the cart is empty, a message is shown with a link to the menu.	Must Have	5
US6	Cart	**Update Cart Quantities** As a customer, I want to increase or decrease item quantities in my cart so I can adjust my order.	1. Each cart item has controls to increase or decrease its quantity. 2. Decreasing quantity to zero removes the item from the cart. 3. Total price updates immediately when quantities change. 4.System shall prevent negative quantities.	Must Have	5
US7	Cart	**Remove Cart Item** As a customer, I want to remove an item from my cart so I no longer order it.	1. A remove button is visible on each cart item. 2. Clicking remove deletes the item immediately without a full page reload. 3. Total price recalculates after removal. 4. If the last item is removed, the cart shows an empty state message.	Must Have	3
US8	Cart	**Cart Badge Count** As a user, I want to see how many items are in my cart from any page so I can	1. A numeric badge is displayed on the cart icon in the navigation bar. 2. Badge shows the current number of distinct items in the cart. 3. Badge is hidden when the count is zero. 4. Count updates on login, logout, or cart changes.	Should Have	3

		keep track of my order.			
US9	Checkout	**Checkout & Payment** As a customer, I want to check out and provide payment information so I can complete my order.	1. A 'Proceed to Checkout' button is available when the cart has items. 2. Checkout flow collects delivery address, payment details, and shows the total. 3. On successful checkout, order is placed and cart is cleared. 4. A success confirmation is shown with order details. 5. Checkout button is disabled or hidden if the cart is empty.	Must Have	8
US10	Checkout	**Add Delivery Address** As a customer, I want to add my delivery address so the restaurant knows where to deliver my order.	1. Address form collects name, phone number, email, and full address. 2. User must be logged in to save an address. 3. A map preview helps the user confirm the location. 4. Success or error feedback is shown after submission. 5. Validation ensures required fields are filled in.	Must Have	5
US11	Feedback	**Submit Feedback** As a customer, I want to leave a review after my order so I can share my experience.	1. System shall allow user to submit feedback 2. Form collects customer's name, rating (1–5 stars), text comment, and an optional image. 3. Feedback is submitted and a confirmation message is shown. 4. Empty or invalid submissions are rejected with clear messages.	Should Have	5
US12	Feedback	**View Testimonials** As a visitor, I want to see customer testimonials so I can check the restaurant's reputation.	1. Testimonials section shows past customer reviews. 2. A slider shows one testimonial at a time with navigation controls. 3. Each testimonial shows customer's name, rating, comment, and image (if provided). 4. A message is shown if no testimonials are available.	Should Have	5
US13	Order Management	**View Order History** As a customer, I want to view my order history so I can track my past purchases.	1. Order history page lists all past orders for the logged-in user. 2. Each order shows order ID, date, items ordered, and total price. 3. User must be logged in to view order history. 4. An empty state is shown if user has no past orders.	Should Have	5

US14	Order Management	**Reorder Previous Order** As a customer, I want to reorder a previous order, i want to do it quickly so that i can purchase again easily.	1.System allows the user to reorder from a past order.. 2.Reordered items are automatically added to the cart. 3.User can modify the cart before confirming the reorder. 4.If an item is unavailable, the system notifies the user. 5.User must be logged in to reorder a previous order. 6.Order is processed as a new order after confirmation.	Should Have	5
US15	Admin	**Admin: View & Manage Orders** As a restaurant admin, I want to view incoming orders so I can manage preparation.	1. Admin dashboard shows all customer orders. 2. Each order shows customer info, items, total amount, and status. 3. Orders are sorted by most recent first. 4. Dashboard is only accessible to admin users.	Should Have	5
US16	Admin	**Admin: Manage Menu Items** As a restaurant admin, I want to manage the menu (add/edit/delete items) so I can keep offerings current.	1. Admin can add new menu items with a name, price, category, and image. 2. Admin can edit existing item details. 3. Admin can delete items from the menu. 4. Changes show up immediately on the customer-facing menu.	Should Have	8
US17	Browsing	**Homepage Carousel** As a visitor, I want to see an engaging homepage with rotating banners so I am encouraged to explore the menu.	1. The carousel auto-plays through slides at regular intervals. 2. Multiple slides show different promotional messages. 3. Each slide has a call-to-action button linking to the menu. 4. The carousel loops continuously.	Could Have	3
US18	Browsing	**Promotional Cards** As a visitor, I want to see current promotions on the homepage so I can take advantage of discounts.	1. Promotion cards are displayed on the homepage. 2. Each card shows a food image, title, and discount percentage. 3. A call-to-action button on each card goes to the menu page.	Could Have	2

US19	Browsing	**Responsive Navigation** As a user, I want a responsive navigation menu that displays as a horizontal navbar on desktop and a hamburger menu on mobile so that I can easily access pages on any screen size.	1. On desktop, navigation links are displayed horizontally. 2. On mobile, a hamburger icon replaces the links and toggles a dropdown menu. 3. The menu closes when a link is clicked. 4. The cart badge count is visible in both desktop and mobile views.	Must Have	5
US20	Browsing	**Search Menu Items** As a customer, I want to search for specific menu items so I can quickly find what I'm looking for.	1. A search icon is available in the navigation bar. 2. Clicking the icon opens a text input field. 3. The system filters and shows matching menu items as the user types or submits. 4. A close button collapses the search bar.	Could Have	5
US21	Browsing	**Browse Menu items** As a customer, I want to view all available menu items so I can decide what to order.	1. All menu items show when the user goes to the menu page. 2. Each item shows its name, image, price, and quantity controls. 3. Items are displayed in a card based grid that adapts to screen size. 4. If no items are available, a clear message is shown.	Must Have	5
US22	Browsing	**Filter Menu by Category** As a customer, I want to filter menu items by category so I can quickly find the food I want.	1. Category filter options are shown (e.g., All, Burger, Pizza, Pasta, Appetizers). 2. Picking a category updates the displayed items right away. 3. The 'All' category shows every item. 4. The active category is visually highlighted.	Should Have	3
US23	Browsing	**View Restaurant Information** As a visitor, I want to learn about the restaurant so I can understand its story and decide to order.	1. About section shows restaurant description and story. 2. An image representing the restaurant is shown. 3. Content is accessible from a navigation link. 4. Content is clear, welcoming, and well-formatted.	Could Have	2

Total Backlog: 107 story points across 23 user stories

2.2 Sprint 1 Planning

Sprint Goal: Deliver the core ordering flow including authentication, menu browsing, cart management and checkout.

Sprint Duration: 2 weeks **Team Size:** 3 members

Sprint 1: 13 Selected User Stories:

ID	Story Title	Epic	Priority	Story Points
US1	User Registration	Auth	Must Have	5
US2	User Login	Auth	Must Have	5
US3	User Logout	Auth	Must Have	2
US4	Add Item to Cart	Cart	Must Have	8
US5	View Shopping Cart	Cart	Must Have	5
US6	Update Cart Quantities	Cart	Must Have	5
US7	Remove Cart Item	Cart	Must Have	3
US8	Cart Badge Count	Cart	Should Have	3
US9	Checkout & Payment	Checkout	Must Have	8
US10	Add Delivery Address	Checkout	Must Have	5
US19	Responsive Navigation	Browsing	Must Have	5
US21	Browse Menu items	Browsing	Must Have	5
US22	Filter Menu by Category	Browsing	Must Have	3

Total	**62**

2.3 Sprint Backlog — Task Breakdown with Estimated Hours

Each user story selected for Sprint 1 is broken down into development tasks with estimated hours. Tasks are sized between 1 to 16 hours as recommended by Scrum guidelines. Team members self assigned tasks during sprint planning.

Technical Setup : Create core tables ,Configure database connection : 6-10hrs

User Story	Task	Est. Hours
Technical Setup **(assigned to team)**	Design database schema (users, menu_items, orders, order_items, cart, addresses, feedback tables)	3
	Create MySQL database and tables via phpMyAdmin/XAMPP	2
	Configure database connection in C#	2
	Set up environment variables (.env) and project structure	1
US-01: User Registration	Design signup form UI component	4
	Implement form validation (email format, password strength)	3
	Build registration API endpoint	4
	Store new user in database	1
	Implement duplicate email check logic	2

	Display success/error feedback messages	2
	Write acceptance tests for registration	2
US-02: User Login	Design login form UI component	3
	Build login API endpoint with credential verification	4
	Implement session management after login	3
	Display error messages for invalid credentials	2
	Redirect user to previous page after login	2
	Write acceptance tests for login	2
US-03: User Logout	Add logout button (visible only when authenticated)	2
	Clear session and reset application state on logout	2
	Reset cart badge to zero	1
	Restore login option in navigation	1
	Write acceptance test for logout	2
US-04: Browse & Navigate	Design menu item card component	4

	Build menu listing API endpoint	4
	Implement responsive grid layout	3
	Handle empty menu state	1
	Write acceptance tests for menu display	2
US-05: Filter Menu by Category	Design category filter buttons UI	3
	Implement client-side filtering logic	3
	Highlight active category visually	1
	Ensure 'All' category displays every item	1
	Write acceptance test for category filtering	2
US-06: Add Item to Cart	Implement 'Add to Cart' button with quantity selector	4
	Enforce login prerequisite	3
	Build add-to-cart API endpoint	3
	Store cart Item in database	1
	Handle duplicate item (update quantity)	3

	Display success/error confirmation alerts	2
	Write acceptance test for add to cart	2
US-07: View Shopping Cart	Design cart page layout	4
	Build cart items API endpoint	3
	Retrieve cart items from database	1
	Display item details (name, image, price, quantity, subtotal)	3
	Calculate and display total price	2
	Handle empty cart state with link to menu	2
	Write acceptance test for cart display	2
US-08: Update Cart Quantities	Add increment/decrement controls to cart items	3
	Build update quantity API endpoint	3
	Auto-remove item when quantity reaches zero	2
	Recalculate total price on change	2
	Write acceptance test for quantity updates	2

US-09: Remove Cart Item	Add remove button to each cart item	2
	Build delete cart item API endpoint	2
	Update cart display without page reload	2
	Recalculate total and handle empty state	2
	Write acceptance test for item removal	2
US-10: Checkout & Payment	Design checkout button and payment modal	4
	Build checkout/order creation API endpoint	6
	Store ordered items in database	2
	Clear cart after successful checkout	2
	Display order confirmation with details	3
	Disable checkout when cart is empty	1
	Write acceptance test for checkout flow	2`
US-11: Add Delivery Address	Design address form (name, phone, email, address)	4

	Build save-address API endpoint	3
	Integrate map preview for location confirmation	4
	Implement form validation for required fields	2
	Display success/error feedback	1
	Write acceptance test for address form	2
US-20:Responsive Navigation	Design desktop horizontal navbar	3
	Implement hamburger icon and mobile dropdown menu	4
	Auto-close menu on link click	1
	Ensure cart badge visible in both views	2
	Write acceptance test for responsive navbar	2
US-22: Cart Badge Count	Add numeric badge to cart icon in navbar	2
	Build cart count API endpoint	2
	Hide badge when count is zero	1
	Update badge on login/logout/cart changes	2

	Write acceptance test for cart badge	2
	Total Estimated Hours	**188**

Part 03 — Requirement Documentation

3.1 System Need Overview

The requirements in this section are derived from the problem context and objectives established in Part 01. The Tasty Bites platform must support two main user groups with distinct needs: customers, who require a reliable way to browse, order, and track food deliveries, and administrators, who require tools to manage menu content and oversee order activity.
From a system perspective, these needs translate into three categories of requirements:

3.2 Functional Requirements

1. User Registration: The system shall let new users create an account using an email and password. Duplicate emails shall be rejected.
2. User Login / Logout: The system shall check user credentials and let them log in securely, and allow them to end their session at any time.
3. Menu Browsing: The system shall show all available menu items with name, image, price, and quantity controls.
4. Category Filtering: The system shall let users filter menu items by food category (for example, Burger, Pizza, Pasta, Appetizers).
5. Cart Management: The system shall let logged in users add items to a cart, change quantities, remove items, and see the total price.
6. Checkout and Payment: The system shall let users go to checkout, enter payment information, provide a delivery address, and place an order that clears the cart.
7. Delivery Address: The system shall let users add a delivery address (name, phone, email, address) with a map preview during checkout.
8. Feedback Submission: The system shall let customers submit a review with a name, star rating (1 to 5), comment, and optional image after checkout.
9. Testimonials Display: The system shall show customer testimonials in a slider with navigation controls.

10. Order History: The system shall let logged-in users view and reorder from their past
orders, showing order ID, date, items, and total price.
11. Admin Menu Management: The system shall let administrators add, edit, and delete menu items. Changes shall show up right away on the customer menu.
12. Admin Order Management: The system shall let administrators view all incoming orders with customer info, items, total, and status.
13. Search: The system shall let users search for menu items by name using a search bar in the navigation.
14. Navigation: The system shall provide a navigation bar that lets users move around the app easily, with a hamburger menu on mobile devices.
15. Cart Badge: The system shall show a badge on the cart icon with the number of items, updated in real time.

3.3 Non-Functional Requirements

1. Performance: The system shall ensure fast and responsive page loading under normal usage conditions.The system shall handle multiple users efficiently without performance degradation

2. Security: User passwords shall be hashed before being stored. The system shall not show sensitive data in error messages.The system shall validate and sanitize all user inputs to prevent security vulnerabilities.
3. Usability: The interface shall be easy to use and require no training. Forms shall give clear feedback when something is wrong.
4. Responsiveness: The application shall work properly on desktop (1024px and above), tablet (768px and above), and mobile (320px and above) screen sizes.
5. Reliability: The system shall handle server errors gracefully and show friendly error messages instead of crashing.
6. Maintainability: Code shall follow a modular, component-based structure. Frontend and backend shall be clearly separated.
7. Scalability: The database design shall support adding more menu categories, items, and users without needing structural changes.

3.4 High-Level System Architecture

Tasty Bites follows a three-tier client-server architecture:
Presentation Layer (Frontend): React.js single-page application with component-based UI, client-side routing, and responsive CSS.
• **Application Layer (Backend):**C# with ASP.NET Core providing RESTful API endpoints for authentication, menu, cart, orders, feedback, and admin operations

• **Data Layer (Database):** MySQL relational database storing users, menu items, orders,order items, addresses, and feedback.

Part 04 — Tool Setup & Configuration

4.1 GitHub Repository

Repo Link:
https://github.com/rosyantoun/Restaurant-ordering-delivery-system.git

4.2 Scrum Board Configuration

Board Link:
https://marilyneantoun06.atlassian.net/jira/software/projects/SCRUM/boards/1

Part 05 — Cost & Effort Estimation

Sprint 1 effort was estimated using a detailed task breakdown, where each user story was split into individual tasks with specific hour estimates. Sprint 2 effort was estimated at a higher level using story points and then converted to hours based on Sprint 1 results.
Using Sprint 1 as a reference: the sprint had 62 story points and 188 estimated hours, giving roughly 1 story point ≈ 3 hours.
Sprint 2 has 43 story points. Using the conversion: 43 × 3 = ~130 hours. With a 25% buffer for uncertainty: ~160 hours.

Sprint 1 Effort Breakdown by Role

The 188 Sprint 1 hours are distributed across four areas of work: frontend development (React components, styling, and client-side logic), backend development (ASP.NET Core API endpoints, authentication, and session handling), database work (MySQL schema, queries, and storage operations), and testing (acceptance tests written for each user story).

Category	Description	Hours	% of Sprint 1
Frontend	React UI components, forms, styling, client-side logic and validation	101	54%

Backend	ASP.NET Core API endpoints, authentication, session management, and server-side logic implemented in C#	51	27%
Database	MySQL schema design, table creation, connection setup, storage queries	10	5%
Testing	Acceptance tests written per user story to validate expected behaviour	26	14%
Total		**188**	**100%**

Effort Estimation

Phase	**Activities**	**Estimated Effort (Hours)**
Planning	Vision, backlog creation, sprint planning	30–40
Sprint 1	Core features: authentication, menu, cart, checkout	188
Sprint 2	Advanced features: feedback, order history, admin dashboard	130–160
Documentation	Diagrams, report writing.	20–25
Total		**368–413 hrs**

Cost Estimation

Assuming a junior developer rate of $10/hour for educational estimation purposes.

Resource	**Hours**	**Rate**	**Cost**
Development (3 devs)	318–348	$10/hr	$3180–$3480
Documentation & Testing	20–25	$10/hr	$200–$250
Project Management	30–40	$10/hr	$300–$400
Total			**$3680–$4130**

Tools Cost

Tool	Purpose	Cost
GitHub	Version control	Free
Jira	Scrum board & sprint management	Free plan
VS Code	Development IDE	Free
MySQL	Database (via XAMPP/phpMyAdmin)	Free
Draw.io	System diagrams	Free
Planning Poker	Sprint estimation	Free
Total		**$0**